Aeza x Shopify

Scopes, endpoints, headers and payloads for the whole commerce flow

Every call between Aeza and a brand's Shopify store: what permissions we hold and why, every Shopify endpoint and webhook we use (product sync, checkout, order creation, tracking), and the Aeza-internal APIs that connect them, each with its **headers, request and JSON response**.

Responses marked **Real** were captured from Sands of India's live store (`cb0c82-9b.myshopify.com`) on 24 Oct 2026 with read-only calls. Payloads marked **Example** follow Shopify's documented format, shortened to the fields we read. Secrets and tokens are never shown.

Item	Value
Shopify API	Admin **GraphQL** API, version **2026-07** (no REST)
Endpoint	`POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json`: one URL for every Admin call
Connector (Shopify gateway)	`https://aeza-partners.vercel.app` (repo `aeza-test-connector`)
Main backend	`https://beta-api.aeza.in` (repo `Aeza-V1-backend`, ECS)

Contents

1. The flow and what runs when
2. Access scopes
3. Authentication (tokens and app-to-app keys)
4. Product sync endpoints
5. Checkout: live price and stock
6. Order creation
7. Order tracking
8. Brand dashboard queries
9. Webhooks (Shopify -> connector)
10. Aeza internal APIs (backend <-> connector)
11. Errors and retries

1. The flow and what runs when

End to end

```
INSTALL            Brand installs its private Aeza app -> token exchange -> offline token stored (encry
   pted)
    |
PRODUCT SYNC       bulkOperationRunQuery -> Shopify builds JSONL -> bulk_operations/finish webhook
    |              -> download JSONL -> ShopifyProduct / ShopifyVariant tables
    |              then live: products/*, inventory_levels/update webhooks -> re-read that product / st
   ock
    |
CHECKOUT           Backend -> connector POST /api/catalog/resolve -> nodes(ids) live price + stock
    |
PAYMENT            Cashfree (backend) -> PAID
    |
ORDER              Backend -> connector POST /api/orders -> tag lookup -> orderCreate (PAID, tag "Aeza"
   )
    |
TRACKING           fulfillments/*, fulfillment_events/create, orders/updated|cancelled webhooks
                   -> connector -> signed event -> backend -> GET /api/order-fulfillments (live)
```

Schedule	What	Where
Real time (seconds)	Webhooks: product, stock, order, fulfilment, delivery events, uninstall	Shopify -> connector
On every quote/checkout	Live price + stock of the cart's variants	backend -> connector -> Shopify
Right after payment	Create the paid Shopify order (retries with backoff on failure)	backend -> connector -> Shopify
Every 60 s	Backend reconciler: re-checks open payments, retries due order steps, polls delivery status	backend
About every hour	Store access token refresh (expiring offline tokens)	connector -> Shopify OAuth
Nightly 02:00 IST	Full catalog re-sync of every connected store (needs `CRON_SECRET`)	Vercel cron -> connector

2. Access scopes

Requested in each brand app's `shopify.app.<slug>.toml` and approved by the brand at install. Granted to Sands of India today: `read_products,read_inventory,read_locations,write_orders,read_fulfillments` (Shopify lists `write_orders` in place of `read_orders`: write includes read).

Scope	Why we need it	Used by	Needed?
`read_products`	Read products, variants, images, collections; product webhooks	Product sync, checkout resolve, dashboard, `products/*` webhooks	**Yes**
`read_inventory`	Stock per variant; inventory webhook	Sync, resolve, `inventory_levels/update`	**Yes**
`read_orders`	Find our order by tag; read fulfilments; order webhooks	Order create (dedupe), tracking, `orders/*` webhooks	**Yes** (included in `write_orders`)
`write_orders`	Create the paid order	`orderCreate`	**Yes**
`read_fulfillments`	Fulfilment and delivery-event webhooks	`fulfillments/*`, `fulfillment_events/create`	**Yes**
`read_locations`	Location details	**Nothing in the code today**	No. Remove, or keep for future per-location stock

Also required (not a scope)

- **Protected customer data** (Level 2: name, email, phone, address) approved in the Dev Dashboard for each brand app. Without it Shopify refuses `orderCreate` with customer details and the order webhooks.
- **Offline access token.** `orderCreate` only works with offline tokens (we use expiring offline tokens).

Consider later

- `read_all_orders`: Shopify only returns orders from the **last 60 days** without it. Needed only if we must look up or track orders older than 60 days (e.g. late returns).
- `write_fulfillments` / `write_returns`: only if Aeza ever fulfils or processes returns on the brand's behalf (today the brand does).

3. Authentication

3.1 Install: token exchange (once per install, handled by the Shopify library)

Request

```
POST https://cb0c82-9b.myshopify.com/admin/oauth/access_token
Content-Type: application/json
Accept: application/json
```

Body

```
{
  "client_id": "<brand app client id>",
  "client_secret": "<brand app client secret>",
  "grant_type": "urn:ietf:params:oauth:grant-type:token-exchange",
  "subject_token": "<App Bridge session token (JWT) from the embedded app>",
  "subject_token_type": "urn:ietf:params:oauth:token-type:id_token",
  "requested_token_type": "urn:shopify:params:oauth:token-type:offline-access-token",
  "expiring": "1"
}
```

Response (shape; values redacted)

```
{
  "access_token": "shpca_...",
  "scope": "read_products,read_inventory,read_locations,write_orders,read_fulfillments",
  "expires_in": 3600,
  "refresh_token": "...",
  "refresh_token_expires_in": 7776000
}
```

Stored **encrypted** (AES-256-GCM) in the connector database, table `Session`, row `offline_<shop>`. The client secret is stored encrypted in `ShopifyStore`.

3.2 Token refresh (automatic, about every hour)

Request

```
POST https://cb0c82-9b.myshopify.com/admin/oauth/access_token
Content-Type: application/json
```

Body

```
{
  "client_id": "<client id>",
  "client_secret": "<client secret>",
  "grant_type": "refresh_token",
  "refresh_token": "<stored refresh token>"
}
```

Response has the same shape as 3.1. **The refresh token rotates every time**, so the new pair must be saved, which the connector does.

3.3 Every Admin API call

Headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Body

```
{
  "query": "<GraphQL document>",
  "variables": {
    "...": "..."
  }
}
```

> A GraphQL document written as `#graphql query X { ... }` on **one line** is sent as an empty document, because `#` starts a comment. This caused the first order failure on 23 Oct (fixed in commit `d785321`). Keep `#graphql` on its own line.

3.4 Aeza app-to-app keys (not Shopify)

Direction	Header	Secret
Backend -> connector	`Authorization: Bearer <key>`	`MAIN_BACKEND_API_KEY` (connector) = `CONNECTOR_API_KEY` (backend)
Connector -> backend	`X-Aeza-Signature: <hex HMAC-SHA256 of raw body>`	`CONNECTOR_EVENTS_SECRET` on both
Cashfree -> backend	`x-webhook-signature` + `x-webhook-timestamp`	Cashfree client secret

4. Product sync endpoints

Code: `app/services/product-sync.server.ts` (connector).

4.1 Start the full catalog export: bulkOperationRunQuery

What for	When it runs
Export every product with variants, images, stock in the background (any catalog size)	On install, nightly at 02:00 IST, or manually

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
mutation StartProductBulkSync($query: String!) {
  bulkOperationRunQuery(query: $query) {
    bulkOperation { id status }
    userErrors { field message }
  }
}
```

Variables (body field "variables")

```
{
  "query": "{ products { edges { node { id title handle descriptionHtml vendor productType tags status u
    pdatedAt media { edges { node { ... on MediaImage { image { url } } } } } variants { edges { node {
    id sku title price compareAtPrice inventoryQuantity selectedOptions { name value } inventoryItem { i
    d } } } } } } } }"
}
```

Real response (Sands of India)

```
{
  "data": {
    "bulkOperationRunQuery": {
      "bulkOperation": {
        "id": "gid://shopify/BulkOperation/8559413952792",
        "status": "CREATED"
      },
      "userErrors": []
    }
  }
}
```

4.2 Get the finished export: node(BulkOperation)

What for	When it runs
Read status and the download URL of the JSONL file	When the bulk_operations/finish webhook arrives

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query BulkOperationResult($id: ID!) {
  node(id: $id) {
    ... on BulkOperation { status errorCode url createdAt }
  }
}
```

Variables (body field "variables")

```
{
  "id": "gid://shopify/BulkOperation/8559413952792"
}
```

Real response (Sands of India)

```
{
  "data": {
    "node": {
      "status": "COMPLETED",
      "errorCode": null,
      "url": "https://storage.googleapis.com/shopify-tiers-assets-prod-us-east1/…(signed, expires in ~7
    days)",
      "createdAt": "2026-10-23T08:57:53Z"
    }
  }
}
```

4.3 The export file (JSONL, downloaded from `url`)

One JSON object per line: a product, then one line per image and per variant, linked by `__parentId`.

Real lines (shortened)

```
{"id":"gid://shopify/Product/9448278884632","title":"Sands Of India Women's White Yoke Embroidered Popli
    n Midi Dress - 1568","handle":"detailed-embroidered-yoke-and-tassels-dress","descriptionHtml":"<div
    ...>...","vendor":"Sands of India","productType":"Dress","tags":["SOIVODRS04/1569/1568"],"status":"A
    CTIVE","updatedAt":"2026-10-21T08:45:56Z"}
{"image":{"url":"https://cdn.shopify.com/s/files/1/0880/0554/2168/files/1568.png?v=1784615566"},"__paren
    tId":"gid://shopify/Product/9448278884632"}
{"id":"gid://shopify/ProductVariant/...","sku":"SIDR0030324XS","title":"XS","price":"1908.00","compareAt
    Price":"4299.00","inventoryQuantity":10,"selectedOptions":[{"name":"Size","value":"XS"}],"inventoryI
    tem":{"id":"gid://shopify/InventoryItem/..."},"__parentId":"gid://shopify/Product/9448278884632"}
```

4.4 Re-read one product: product(id)

What for	When it runs
Refresh one product and all its variants after a change (never trust the webhook body)	products/create and products/update webhooks

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query RefreshProduct($id: ID!, $variantsAfter: String) {
  product(id: $id) {
    id title handle descriptionHtml vendor productType tags status updatedAt
    media(first: 50) { nodes { ... on MediaImage { image { url } } } }
    variants(first: 250, after: $variantsAfter) {
      nodes {
        id sku title price compareAtPrice inventoryQuantity
        selectedOptions { name value }
        inventoryItem { id }
      }
      pageInfo { hasNextPage endCursor }
    }
  }
}
```

Variables (body field "variables")

```
{
  "id": "gid://shopify/Product/9704666956056",
  "variantsAfter": null
}
```

Real response (shortened to 2 images / 2 variants)

```
{
  "data": {
    "product": {
      "id": "gid://shopify/Product/9704666956056",
      "title": "Sands Of India Women's Indigo Breeze Ruffle Top- 2997",
      "handle": "ndigo-breeze-ruffle-top-soivolt33",
      "descriptionHtml": "<p>This Sands Of India indigo top features ruffled sleeves with ladder…",
      "vendor": "Sands of India",
      "productType": "Top",
      "tags": [
        "SOIVOLT46/2998/2997"
      ],
      "status": "ACTIVE",
      "updatedAt": "2026-10-23T15:12:45Z",
      "media": {
        "nodes": [
          {
            "image": {
              "url": "https://cdn.shopify.com/s/files/1/0880/0554/2168/files/2997_1.png?v=1785220790"
            }
          },
          {
            "image": {
              "url": "https://cdn.shopify.com/s/files/1/0880/0554/2168/files/210302_11.jpg?v=1743581282"
            }
          }
        ]
      },
      "variants": {
        "nodes": [
          {
            "id": "gid://shopify/ProductVariant/49946819789080",
            "sku": "SIRT0330125XS",
            "title": "XS",
            "price": "995.00",
            "compareAtPrice": "1999.00",
            "inventoryQuantity": 1,
            "selectedOptions": [
              {
                "name": "Size",
                "value": "XS"
              }
            ],
            "inventoryItem": {
              "id": "gid://shopify/InventoryItem/52000240697624"
            }
          },
          {
            "id": "gid://shopify/ProductVariant/49946819821848",
            "sku": "SIRT03301250S",
            "title": "S",
            "price": "995.00",
            "compareAtPrice": "1999.00",
            "inventoryQuantity": 5,
            "selectedOptions": [
              {
                "name": "Size",
                "value": "S"
              }
            ],
            "inventoryItem": {
              "id": "gid://shopify/InventoryItem/52000240730392"
            }
          }
        ],
        "pageInfo": {
          "hasNextPage": true,
          "endCursor": "eyJsYXN0X2lkIjo0OTk0NjgxOTgyMTg0OCwibGFzdF92YWx1ZSI6IjIifQ=="
        }
      }
    }
  }
}
```

4.5 Re-read one variant's stock: inventoryItem(id)

What for	When it runs
Update availableQty after a stock change	inventory_levels/update webhook

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query InventoryItemQuantity($id: ID!) {
  inventoryItem(id: $id) {
    variants(first: 1) { nodes { id inventoryQuantity } }
  }
}
```

Variables (body field "variables")

```
{
  "id": "gid://shopify/InventoryItem/52000240763160"
}
```

Real response (Sands of India)

```
{
  "data": {
    "inventoryItem": {
      "variants": {
        "nodes": [
          {
            "id": "gid://shopify/ProductVariant/49946819854616",
            "inventoryQuantity": 6
          }
        ]
      }
    }
  }
}
```

5. Checkout: live price and stock

5.1 Live variant price and stock: nodes(ids)

What for	When it runs
Authoritative price and stock of the variants in the cart, read at that moment	Every cart quote and checkout (via connector POST /api/catalog/resolve)

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query LiveVariants($ids: [ID!]!) {
  nodes(ids: $ids) {
    ... on ProductVariant {
      id price compareAtPrice inventoryQuantity inventoryPolicy
      inventoryItem { tracked }
    }
  }
}
```

Variables (body field "variables")

```
{
  "ids": [
    "gid://shopify/ProductVariant/49946819854616"
  ]
}
```

Real response (Sands of India)

```
{
  "data": {
    "nodes": [
      {
        "id": "gid://shopify/ProductVariant/49946819854616",
        "price": "995.00",
        "compareAtPrice": "1999.00",
        "inventoryQuantity": 6,
        "inventoryPolicy": "DENY",
        "inventoryItem": {
          "tracked": true
        }
      }
    ]
  }
}
```

Stock rule: `null` (unlimited) when `tracked` is false or `inventoryPolicy` is `CONTINUE`; otherwise `inventoryQuantity`.

6. Order creation

Code: `app/services/orders.server.ts` (connector). Called by the backend after Cashfree confirms the payment. Idempotent per Aeza order id: a database claim plus the Shopify tag lookup below.

6.1 Store currency: shop

What for	When it runs
Currency for prices and the transaction	Before every order

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query ShopCurrency { shop { currencyCode } }
```

Real response (Sands of India)

```
{
  "data": {
    "shop": {
      "currencyCode": "INR"
    }
  }
}
```

6.2 Already created? orders(query: tag)

What for	When it runs
Exactly-once: find an order already tagged with this Aeza order id (e.g. after a timeout)	Before every orderCreate

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query FindAezaOrder($q: String!) {
  orders(first: 1, query: $q) { nodes { id name } }
}
```

Variables (body field "variables")

```
{
  "q": "tag:\"AZ-20261023-C93BQB\""
}
```

Real response (Sands of India)

```
{
  "data": {
    "orders": {
      "nodes": [
        {
          "id": "gid://shopify/Order/7461612945688",
          "name": "#1829"
        }
      ]
    }
  }
}
```

6.3 Create the paid order: orderCreate

What for	When it runs
Create the PAID order in the brand's store, decrement stock	Once per paid Aeza order (retried with backoff until it succeeds, else refund)

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Mutation

```
mutation CreateAezaOrder($order: OrderCreateOrderInput!, $options: OrderCreateOptionsInput) {
  orderCreate(order: $order, options: $options) {
    order { id name }
    userErrors { field message code }
  }
}
```

Variables (customer details are placeholders; ids, amounts and payment ref are from order #1829)

```
{
  "order": {
    "currency": "INR",
    "email": "customer@example.com",
    "phone": "+919800000000",
    "test": false,
    "financialStatus": "PAID",
    "tags": [
      "Aeza",
      "AZ-20261023-C93BQB"
    ],
    "note": "Aeza order AZ-20261023-C93BQB (payment 6815230204)",
    "sourceName": "Aeza",
    "sourceIdentifier": "AZ-20261023-C93BQB",
    "customAttributes": [
      {
        "key": "aeza_order_id",
        "value": "AZ-20261023-C93BQB"
      }
    ],
    "lineItems": [
      {
        "variantId": "gid://shopify/ProductVariant/49946819854616",
        "quantity": 1,
        "priceSet": {
          "shopMoney": {
            "amount": "995.00",
            "currencyCode": "INR"
          }
        }
      }
    ],
    "shippingAddress": {
      "firstName": "Asha",
      "lastName": "Rao",
      "address1": "12 MG Road",
      "city": "Bengaluru",
      "provinceCode": "KA",
      "zip": "560068",
      "countryCode": "IN",
      "phone": "+919800000000"
    },
    "transactions": [
      {
        "kind": "SALE",
        "status": "SUCCESS",
        "gateway": "Aeza (Cashfree)",
        "authorizationCode": "6815230204",
        "amountSet": {
          "shopMoney": {
            "amount": "995.00",
            "currencyCode": "INR"
          }
        }
      }
    ]
  },
  "options": {
    "inventoryBehaviour": "DECREMENT_OBEYING_POLICY",
    "sendReceipt": false,
    "sendFulfillmentReceipt": false
  }
}
```

Test orders (sandbox payments) send `"test": true` and the extra tag `Aeza Test`. A shipping fee adds `shippingLines: [{ title: "Aeza shipping", priceSet }]` and is included in the transaction amount.

Real result (order #1829, created 23 Oct 2026 20:42 IST)


```
{
  "data": {
    "orderCreate": {
      "order": {
        "id": "gid://shopify/Order/7461612945688",
        "name": "#1829"
      },
      "userErrors": []
    }
  }
}
```


Example: refusal (no order is created; the payment is refunded automatically)


```
{
  "data": {
    "orderCreate": {
      "order": null,
      "userErrors": [
        {
          "field": [
            "order",
            "lineItems",
            "0",
            "quantity"
          ],
          "message": "Not enough inventory available",
          "code": "INVENTORY_..."
        }
      ]
    }
  }
}
```

7. Order tracking

7.1 Fulfilments and delivery events: order(id)

What for	When it runs
Tracking numbers, carrier, delivery status timeline, cancellation	After a fulfilment/cancel webhook, and polled by the backend reconciler for shipped orders

Request line and headers

```
POST https://cb0c82-9b.myshopify.com/admin/api/2026-07/graphql.json
Content-Type: application/json
X-Shopify-Access-Token: <store access token>   # per brand, from the connector, never logged
```

Query (body field "query")

```
query AezaOrderFulfillments($id: ID!) {
  order(id: $id) {
    cancelledAt
    fulfillments(first: 10) {
      id displayStatus createdAt inTransitAt estimatedDeliveryAt deliveredAt
      trackingInfo { company number url }
      events(first: 50) { nodes { status happenedAt message } }
    }
  }
}
```

Variables (body field "variables")

```
{
  "id": "gid://shopify/Order/7461612945688"
}
```

Real response (order #1829, not shipped yet)

```
{
  "data": {
    "order": {
      "cancelledAt": null,
      "fulfillments": []
    }
  }
}
```

Example: after the brand ships


```
{
  "data": {
    "order": {
      "cancelledAt": null,
      "fulfillments": [
        {
          "id": "gid://shopify/Fulfillment/1234567890",
          "displayStatus": "IN_TRANSIT",
          "createdAt": "2026-10-24T06:10:00Z",
          "inTransitAt": "2026-10-24T09:00:00Z",
          "estimatedDeliveryAt": "2026-10-27T18:00:00Z",
          "deliveredAt": null,
          "trackingInfo": [
            {
              "company": "Delhivery",
              "number": "1234567890123",
              "url": "https://www.delhivery.com/track/package/1234567890123"
            }
          ],
          "events": {
            "nodes": [
              {
                "status": "IN_TRANSIT",
                "happenedAt": "2026-10-24T09:00:00Z",
                "message": null
              }
            ]
          }
        }
      ]
    }
  }
}
```


8. Brand dashboard queries

Used by the Aeza app inside the brand's Shopify admin (product picker). Code: `app/models/products.server.ts`.

Query	Purpose
`products(first: 20, after, sortKey: CREATED_AT, reverse: true)`	Product grid, 20 per page
`collection(id) { products(first, after) }`	Product grid filtered by collection
`nodes(ids) { ... on Product }`	Re-read products before saving the brand's selection
`collections(first: 100, sortKey: TITLE) { productsCount }`	Collection filter dropdown
`productsCount` / `collection { productsCount }`	Totals in the dashboard header

Real response: productsCount


```
{
  "data": {
    "productsCount": {
      "count": 224
    }
  }
}
```

Real response: collections (first 2)


```
{
  "data": {
    "collections": {
      "nodes": [
        {
          "id": "gid://shopify/Collection/494088257816",
          "title": "40% Off Collection",
          "productsCount": {
            "count": 16
          }
        },
        {
          "id": "gid://shopify/Collection/492626313496",
          "title": "Aesthetic Allure",
          "productsCount": {
            "count": 36
          }
        }
      ]
    }
  }
}
```

9. Webhooks (Shopify -> connector)

Declared per brand app in `shopify.app.<slug>.toml`, pushed with `shopify app deploy --config <slug>`. Shopify sends `POST` with a JSON body (REST-style, snake_case).

Headers on every webhook

Header	Use
`X-Shopify-Hmac-Sha256`	Base64 HMAC-SHA256 of the raw body with the brand app's client secret: verified first
`X-Shopify-Shop-Domain`	Which store, so we know which app's secret to verify with
`X-Shopify-Topic`	e.g. `orders/cancelled`
`X-Shopify-Webhook-Id`	Unique per delivery: stored in `WebhookEvent`, so repeats are ignored
`X-Shopify-API-Version`	`2026-07`
`X-Shopify-Event-Id`, `X-Shopify-Triggered-At`	Event id / time (informational)

Topics

Topic	Sent to	What we do
`products/create`, `products/update`	`/webhooks/events`	Re-read product (4.4), upsert
`products/delete`	`/webhooks/events`	Mark deleted, unpublish
`inventory_levels/update`	`/webhooks/events`	Re-read stock (4.5)
`orders/updated`, `orders/cancelled`	`/webhooks/events`	Update our order; forward fulfilled/cancelled to backend
`fulfillments/create`, `fulfillments/update`	`/webhooks/events`	Save tracking; forward to backend
`fulfillment_events/create`	`/webhooks/events`	Forward delivery event to backend
`bulk_operations/finish`	`/webhooks/events`	Import the catalog export (4.2, 4.3)
`app/uninstalled`	`/webhooks/app/uninstalled`	Store uninstalled, unpublish all, delete tokens
`app/scopes_update`	`/webhooks/app/scopes_update`	Update granted scopes
`customers/data_request`, `customers/redact`, `shop/redact`	`/webhooks/compliance`	GDPR; `shop/redact` deletes the shop's data

Example payloads (shortened to the fields we read)

products/create, products/update (we only use admin_graphql_api_id and re-read the product)

```
{
  "admin_graphql_api_id": "gid://shopify/Product/9704666956056",
  "id": 9704666956056,
  "title": "...",
  "updated_at": "2026-10-24T05:30:00+05:30"
}
```

products/delete

```
{
  "id": 9704666956056
}
```

inventory_levels/update

```
{
  "inventory_item_id": 52000240763160,
  "location_id": 81234567890,
  "available": 6,
  "updated_at": "2026-10-24T05:30:00+05:30"
}
```

orders/updated, orders/cancelled

```
{
  "admin_graphql_api_id": "gid://shopify/Order/7461612945688",
  "id": 7461612945688,
  "name": "#1829",
  "financial_status": "paid",
  "fulfillment_status": "fulfilled",
  "cancelled_at": null
}
```

fulfillments/create, fulfillments/update

```
{
  "id": 1234567890,
  "order_id": 7461612945688,
  "status": "success",
  "tracking_company": "Delhivery",
  "tracking_number": "1234567890123",
  "tracking_url": "https://www.delhivery.com/track/package/1234567890123",
  "shipment_status": "in_transit"
}
```

fulfillment_events/create

```
{
  "id": 987654321,
  "fulfillment_id": 1234567890,
  "order_id": 7461612945688,
  "status": "in_transit",
  "happened_at": "2026-10-24T09:00:00+05:30"
}
```

bulk_operations/finish

```
{
  "admin_graphql_api_id": "gid://shopify/BulkOperation/8559413952792",
  "status": "completed",
  "type": "query",
  "error_code": null,
  "created_at": "2026-10-23T08:57:53Z",
  "completed_at": "2026-10-23T08:57:58Z"
}
```

Response to every webhook: **HTTP 200 with an empty body** immediately after the HMAC check; the work runs in the background. 401 = bad HMAC, 404 = unknown store.

10. Aeza internal APIs (backend <-> connector)

10.1 POST /api/catalog/resolve (backend -> connector)

Request line and headers

```
POST https://aeza-partners.vercel.app/api/catalog/resolve
Authorization: Bearer <MAIN_BACKEND_API_KEY>
Content-Type: application/json
```

Body

```
{
  "shop": "cb0c82-9b.myshopify.com",
  "items": [
    {
      "handle": "soivocs76-black-poplin-embroidered-co-ord-set-for-women",
      "size": "XS"
    },
    {
      "handle": "cyan-organza-neck-tie-detail-shirt-2816",
      "size": "M"
    }
  ]
}
```

Real response, 200 (sizes shortened to one product)


```
{
  "items": [
    {
      "handle": "soivocs76-black-poplin-embroidered-co-ord-set-for-women",
      "size": "XS",
      "ok": true,
      "variantId": "gid://shopify/ProductVariant/51615929434392",
      "sku": "SICO0410326XS",
      "title": "Sands Of India Black Poplin Embroidered Co-ord Set for Women - 3104",
      "pricePaise": 235300,
      "compareAtPricePaise": 499900,
      "stock": 13,
      "live": true
    },
    {
      "handle": "cyan-organza-neck-tie-detail-shirt-2816",
      "size": "M",
      "ok": true,
      "variantId": "gid://shopify/ProductVariant/49056966476056",
      "sku": "SISH00610240M",
      "title": "Sands Of India Pink Organza Neck Tie Detail top with Camisole - 2816",
      "pricePaise": 162100,
      "compareAtPricePaise": 379900,
      "stock": 49,
      "live": false
    }
  ],
  "sizes": {
    "soivocs76-black-poplin-embroidered-co-ord-set-for-women": [
      {
        "size": "XS",
        "stock": 13
      },
      {
        "size": "S",
        "stock": 23
      },
      {
        "size": "M",
        "stock": 40
      },
      {
        "size": "L",
        "stock": 54
      },
      {
        "size": "XL",
        "stock": 15
      }
    ]
  }
}
```


Item errors: `{ ok: false, code: PRODUCT_NOT_FOUND | SIZE_REQUIRED | SIZE_NOT_FOUND }`. HTTP errors: 401 unauthorized, 400 invalid_request, 404 unknown_shop, 409 store_not_connected, 502 shopify_unavailable.

10.2 POST /api/orders (backend -> connector)

Request line and headers

```
POST https://aeza-partners.vercel.app/api/orders
Authorization: Bearer <MAIN_BACKEND_API_KEY>
Content-Type: application/json
```

Body (customer details are placeholders)

```
{
  "shop": "cb0c82-9b.myshopify.com",
  "aezaOrderId": "AZ-20261023-C93BQB",
  "isTest": false,
  "lineItems": [
    {
      "variantId": "gid://shopify/ProductVariant/49946819854616",
      "quantity": 1,
      "price": "995.00"
    }
  ],
  "shippingPrice": "0.00",
  "customer": {
    "name": "Asha Rao",
    "email": "customer@example.com",
    "phone": "+919800000000"
  },
  "shippingAddress": {
    "firstName": "Asha",
    "lastName": "Rao",
    "address1": "12 MG Road",
    "city": "Bengaluru",
    "provinceCode": "KA",
    "zip": "560068",
    "countryCode": "IN",
    "phone": "+919800000000"
  },
  "paymentReference": "6815230204"
}
```

201 Created (order #1829)

```
{
  "shopifyOrderId": "gid://shopify/Order/7461612945688",
  "shopifyOrderName": "#1829",
  "created": true
}
```

200 when the same aezaOrderId was already created (no second order)

```
{
  "shopifyOrderId": "gid://shopify/Order/7461612945688",
  "shopifyOrderName": "#1829",
  "duplicate": true
}
```

Status	error	Meaning / backend action
400	`invalid_request`	Bad body (backend alerts, retries slowly)
401	`unauthorized`	Wrong key (alert, retry)
404 / 409	`unknown_shop` / `store_not_connected`	Store not set up or uninstalled (alert, retry)
409	`order_in_progress`	Same id being created right now (retry)
422	`unknown_variant`, `product_not_available`, `out_of_stock`, `shopify_rejected`	Shopify won't take it: **refund**
502	`shopify_unavailable`	Outcome unknown: retry; the tag lookup prevents a duplicate

10.3 GET /api/order-fulfillments (backend -> connector)

Request line and headers

```
GET https://aeza-partners.vercel.app/api/order-fulfillments?shop=cb0c82-9b.myshopify.com&aezaOrderId=AZ-
    20261023-C93BQB
Authorization: Bearer <MAIN_BACKEND_API_KEY>
```

Real response, 200 (order #1829, not shipped yet)

```
{
  "cancelledAt": null,
  "fulfillments": []
}
```

Same shape as section 7 (`fulfillments[]` with `tracking[]` and `events[]`). 404 `not_found` if Aeza has no Shopify order for that id.

10.4 POST /api/webhooks/connector (connector -> backend)

Request line and headers

```
POST https://beta-api.aeza.in/api/webhooks/connector
Content-Type: application/json
X-Aeza-Signature: <hex HMAC-SHA256(raw body, CONNECTOR_EVENTS_SECRET)>
```

Body

```
{
  "topic": "orders/fulfilled",
  "shop": "cb0c82-9b.myshopify.com",
  "shopifyOrderId": "gid://shopify/Order/7461612945688",
  "eventId": "<X-Shopify-Webhook-Id of the original delivery>"
}
```

200 (401 if the signature doesn't match; 5xx makes the connector retry 3 times)

```
{
  "received": true
}
```

Topics forwarded: `orders/fulfilled`, `orders/cancelled`, `fulfillments/create`, `fulfillments/update`, `fulfillment_events/create`. The backend then re-reads tracking via 10.3, or starts the cancel/refund path.

11. Errors and retries

Situation	What happens
Webhook delivered twice	Second copy ignored (`WebhookEvent` keyed by `X-Shopify-Webhook-Id`)
Shopify throttles (`THROTTLED`)	Connector retries up to 5 times with short backoff
Order create fails, outcome unknown	Backend retries: about 30 s, 2 min, 5 min, 15 min, 30 min, 1 h, 2 h, 4 h (~7.5 h), then refunds
Order refused (stock, variant)	No order created; automatic full refund
Token expired	Refreshed automatically before the call (3.2)
Brand uninstalls	Tokens deleted, products unpublished, new orders refused -> refunded
Missed webhook	Backend reconciler (60 s) and nightly full sync catch up